

Review for “Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints”

1 Overall Evaluation

This paper, titled “*Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints*,” addresses the fundamental operational challenge of efficiently serving large language models (LLMs) under tight GPU memory limits. The authors model LLM inference as a multi-stage online scheduling problem, capturing both the prefill and decode phases of token generation and the dynamically growing Key-Value (KV) cache that consumes GPU memory during inference.

To make this problem tractable, the authors develop a fluid dynamics approximation that serves as an analytical benchmark for optimal scheduling under memory constraints. Building on this foundation, they introduce two algorithms:

- **WAIT (Waiting for Accumulated Inference Threshold)**, designed for the idealized setting where output lengths are known, and
- **Nested WAIT**, which generalizes the approach to the realistic case of *unknown output lengths (types)*.

Both algorithms are shown to be asymptotically near-optimal in heavy traffic, achieving strong performance in terms of throughput, latency, and Time-to-First-Token (TTFT). Extensive experiments using Llama-7B on the Vidur inference engine—with both synthetic and real workloads—demonstrate that these methods outperform popular inference systems such as vLLM and Sarathi. The work thus bridges operations research and modern machine learning, combining queueing analysis with practical system design for efficient LLM deployment.

The topic is timely and important, given the increasing need to reduce LLM serving costs and energy consumption while maintaining responsiveness. The proposed fluid-guided approach provides both theoretical insight and practical scheduling principles for practitioners. The theoretical development is solid and well-grounded, leveraging coupling and heavy-traffic analysis to establish asymptotic optimality. The extensions are thorough, and the treatment of the unknown output length setting is insightful, revealing how uncertainty in workload size can be managed dynamically.

Overall, this is a high-quality paper that effectively demonstrates how tools from operations research—particularly fluid modeling and queueing theory—can yield actionable strategies for real-world AI infrastructure. The manuscript is clearly written, technically rigorous, and of broad interest to both the OR and machine learning communities. I have some comments/questions on the modeling, theoretical analysis, and experiments. But in general I think they can be addressed with reasonable revision efforts. The comments are provided below.

2 Modeling

First of all, I highly appreciate that the authors made an effort to illustrate how LLM inference works in an educational way. Figures 1–2 and Section 2 are clearly written and could serve as a valuable pedagogical reference for the broader OR community.

My **main comment** concerns Eq. (1), which models the iteration time:

$$\tau = d_0 + d_1 \cdot \left(\sum_{i=1}^{n_1} k_i + \sum_{i'=1}^{n_2} s_{i'} \right),$$

where k_i denotes the length of prefill prompts for requests in the prefill stage, and $s_{i'} = l_{j(i)} + k_i$ represents the combined length of prefill and processed decoding tokens for the requests in the decoding stage.

My understanding is that the authors intend to capture the KV-cache memory loading cost here. However, a more complete formulation might also include the computation time:

$$\begin{aligned} \tau = & d_0 + d_1 \cdot \underbrace{\left(\sum_{i=1}^{n_1} k_i + \sum_{i'=1}^{n_2} s_{i'} \right)}_{\text{KV-cache loading}} \\ & + \underbrace{O\left(\sum_{i=1}^{n_1} k_i + n_2\right)}_{\text{linear layer}} + \underbrace{O\left(\sum_{i=1}^{n_1} k_i^2\right)}_{\text{attention (prefill)}} + \underbrace{O\left(\sum_{i'=1}^{n_2} s_{i'}\right)}_{\text{attention (decode)}}, \end{aligned}$$

as discussed in [2].

In practice, three asymptotic regimes may arise:

1. Long output or large batch size:

$$\tau \approx d_0 + O\left(\sum_{i'=1}^{n_2} s_{i'}\right).$$

2. Long input context:

$$\tau \approx d_0 + O\left(\sum_{i=1}^{n_1} k_i^2\right).$$

3. Moderate input, batch size, and output lengths: The linear layer term dominates [3]:

$$\tau \approx d_0 + O\left(\sum_{i=1}^{n_1} k_i + n_2\right).$$

See also Figure 4 in [1].

I might be mistaken, but it seems the authors' theoretical analysis primarily targets scenario (1). If so, d_1 may not need to serve as a shared factor in front of both summations, as the dominant component differs across regimes.

I would encourage the authors to explicitly state which scenario their model represents and to discuss its relation to existing studies [2, 3, 1]. Relatedly, Figure 3 could be clarified by specifying the exact numbers of $\{k_i, s_{i'}\}_{i \in [n_1], i' \in [n_2]}$ in each batch and describing the experimental setup.

It would be perfectly acceptable if the authors focus on scenario (1) (large output and/or batch size) and use Eq. (1) for analytical simplicity (so that the memory cost and iteration time share the same formula), as long as this assumption and its implications are clearly stated.

Minor Comments

1. Page 7, lines 40–44: The flow of sentences could be improved for coherence.
2. Page 9, line 15: Since k_i is later used in the decoding stage, perhaps $l_{j(i)}$ or l_i would be a clearer notation.
3. Page 9, line 50: Should d be d_1 ?
4. Page 10, line 42: Can $k_i^t = 0$ indicate the prefill stage? If so, please clarify.
5. Page 10, line 48: A short footnote clarifying that prefill throughput is not considered here would be helpful.
6. Page 11, lines 17–22: Π is defined as non-preemptive policies, yet preemption later appears to be allowed. Please clarify whether preemption is permitted (my understanding is yes, while retaining unevicted KV-cache).

3 Theoretical Analysis

First of all, I find the following points particularly noteworthy:

1. The design of both WAIT and NESTED WAIT is novel and elegant. The construction of WAIT makes the analysis tractable, while NESTED WAIT reflects sharp insight into the algorithm’s execution; the underlying idea is clever and well-motivated.
2. The proofs of Theorems 1 and 2, which rely on coupling arguments, are non-trivial and enjoyable to read.

My **major comments** are as follows:

1. **On the analysis assumption.** The paper assumes

$$C \geq M^*.$$

However, under a fluid limit, this might imply a non-trivial requirement for memory consumption and batch size. For instance, consider a prompt type that produces 1000 output tokens; this implies at least a batch size of 1000 and a KV-cache of order $500K$ tokens even for $n_j = 1$ (i.e., one request per stage), which seems unrealistic in practice. For reference, a 40 GB A100 GPU can maintain at most $\sim 200K$ tokens of KV-cache in **bf16** for **Llama3.1-8B** [4]. It is reasonable to adopt this assumption for theoretical tractability, but I suggest explicitly noting that it is not as benign as it might initially appear. The experiments should also clarify whether this assumption is consistent with their setup.

2. **On the proof of Theorem 1.** I appreciate the proof techniques (e.g., decoupling and probabilistic arguments), but the decoding-stage analysis seems to be missing from the appendix. For example, in

Lemma 1, the authors directly assume

$$W^{t+1} = W^t + X^t - \lambda \mathbf{1}(W^t + X^t \geq \lambda),$$

meaning that once the queue length exceeds λ , λ requests leave the system in the next step. However, if there are l' decoding stages, shouldn't we need to wait for l' batch executions before we have requests completing? The multi-type analysis also does not clarify this. Under the WAIT algorithm (which may be the motivation for waiting until n_j prompts have queued), this might be straightforward, but it would still be helpful to clarify. Currently, this component appears missing in the proof.

3. **Alternative algorithms.** Related to the above, what happens if the algorithm does *not* wait (i.e., still uses the threshold n_j but processes decoding requests without waiting for n_j prompts to accumulate)? It would be valuable to discuss what challenges this would pose for the theoretical analysis and what implications it might have in practice.
4. **On the heavy-traffic assumption.** From the proof, it seems that the heavy-traffic assumption is only invoked near the end. What exact role does it play in the analysis? Could the theorem be stated (or partially proved) without this assumption?

3.1 Minor Comments for Section 3

1. Page 11, line 55: a stylistic suggestion (feel free to ignore) — perhaps drop the subscript j in the single-type fluid model for simplicity.
2. Page 12, line 19: should it be $\lambda_j d_1 (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)$? The same applies to lines 21, 31, and 33 (d_0 should be d_1).
3. Page 12, line 30: “requires” $\rightarrow$ “at least requires”?
4. Page 13, Eq. (4): should it be

$$\frac{\frac{n_j^*}{l'_j+1} l'_j}{d_0 + d_1 M_j^*},$$

since during a period of length $d_0 + d_1 M_j^*$, $\frac{n_j^*}{l'_j+1}$ requests complete, and each generates l'_j decoding tokens?

5. Page 13, line 46: should it be

$$(d_0 + d_1 M^*) \lambda_j = \frac{n_j^*}{l'_j + 1}?$$

6. Page 14, line 20: replace $\sum_{j=1}^n$ with $\sum_{j=1}^m$ and ensure d_1 is included.
7. Page 14, line 23: should it be $\sum_{j=1}^m \lambda_j l'_j$? (each request is left with l'_j decoding tokens). Alternatively, you may wish to redefine throughput accordingly from l'_j to $l'_j + 1$? The same issue appears in lines 49–50.
8. Page 14, Proposition 2: is this simply stating that the expected throughput over $[0, T]$ is bounded by the expected number of decoding tokens arriving during $[0, T]$? If so, it might be worth clarifying explicitly.

3.2 Minor Comments for Sections 4–5 and the Appendix

1. Page 17, line 26: clarify that n_j is denoted to represent $\frac{n_j^*}{l_j^*+1}$ rather than n_j^* itself.
2. Page 17, line 37: where does M^* appear in Eq. (9)?
3. Page 17, line 52: the notation $\Delta T_{[1,\dots,m]}$ is undefined.
4. Page 35, line 4: the citation “Asmussen S, Asmussen S, Asmussen S” duplicates the author’s name.
5. Page 39, line 38: typo — “.Consider” $\rightarrow$ “. Consider”.
6. Page 39, line 40: should read “type 2 with $l_2 = 1$ ”, etc.
7. Page 39, Proposition 4 proof: clarify why “equilibrium batch” represents the optimal situation to make the proof more rigorous.
8. Page 41, line 35: what is the “booking limiting policy”?
9. Page 41, line 41: time step t ?
10. Page 41, Lemma 1: clarify the reduction from the main problem to this lemma’s setting.
11. Page 42, line 35: should it be $\tilde{W}^T \geq W^T + \lambda$ instead of $\tilde{W}^T \geq \lambda^T$?
12. Page 42, Lemma 4: S^k is undefined.
13. Page 43, line 9: the discussion of the conventional heavy-traffic regime could be expanded for completeness.
14. Page 43, line 32: why $\text{Arrival}_j \leq \text{Completion}_j$? The arrival is random, whereas the completion is bounded by n_j , correct?
15. Page 43, line 36: should $M^\pi = M^*$?
16. Page 44, line 14: have you defined W_t and $\tilde{W}_t$ for times t between events (i.e., not multiples of ΔT)?
17. Page 44, line 18: I am not sure I follow the proof in Appendix E.3. Would the following reasoning suffice?

- If $W^{k\Delta} \geq n_j$, then

$$W^{(k+1)\Delta} \leq W^{k\Delta} + X^\Delta - n_j,$$

since a batch including j must be processed at time $(k+1)\Delta$. Thus,

$$\tilde{W}^{(k+1)\Delta} \geq \tilde{W}^{k\Delta} + X^\Delta - n_j \stackrel{(i)}{\geq} W^{k\Delta} + X^\Delta - n_j \geq W^{(k+1)\Delta},$$

where (i) holds by induction.

- If $W^{k\Delta} < n_j$, then $\tilde{W}^{k\Delta} \geq W^{k\Delta} + n_j$, implying

$$\tilde{W}^{(k+1)\Delta} \geq \tilde{W}^{k\Delta} + X^\Delta - n_j \geq W^{k\Delta} + X^\Delta \geq W^{(k+1)\Delta}.$$

This seems to complete the proof.

18. Page 44, line 52: cite Kingman.

4 Experiments

1. **Batch size and memory configuration.** What are the batch size B , the memory bound M^* , and the capacity C used in Section 6.1 (the high-demand case)? If my understanding is correct, even when $n_1 = n_2 = n_3 = 1$, the batch size must satisfy $B \geq 600$, and the corresponding memory requirement is on the order of $O(300^2 + 200^2 + 100^2)$. Please clarify these parameter settings.
2. **Simulation fidelity.** In the above large-batch scenario, the **Vidur** simulation may no longer accurately capture real GPU behavior. It would be informative to validate the results on an actual GPU, such as an NVIDIA A100 (e.g., via Google Colab), to empirically measure batch processing time. I recommend including a figure comparing simulated and real processing times to confirm the validity of the assumptions (and clarify this in the abstract as well if it is using simulation instead of the actual GPU).
3. **Sarathi configuration.** How is SARATHI configured for the high-demand case? Please specify key parameters such as chunk size, reserved memory allocation (to prevent memory overflow), and any other relevant hyperparameters. These details would help readers reproduce the experiments and understand performance differences across baselines.
4. **Clarification for Figure 7.** What does the “Prompt Number” on the right-hand side of Figure 7 represent?
5. **Parameter settings in Section 6.2.** Similar questions apply to Section 6.2: what are the specific values of $n_1, \dots, n_{10}$, and what are the corresponding batch size B , memory bound M^* , and capacity C used in these experiments?
6. **Practical implications.** In the case when the output length exceeds 1000 tokens (or longer), it may be useful to discuss how the algorithm behaves in such regimes. For instance, could a threshold-based algorithm *without waiting* serve as a viable alternative to mitigate memory constraints?

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
- [2] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. Optimal scheduling algorithms for llm inference: Theory and practice. *arXiv preprint arXiv:2508.01002*, 2025.
- [3] Yueying Li, Jim Dai, and Tianyi Peng. Throughput-optimal scheduling algorithms for llm inference and ai agents. *arXiv preprint arXiv:2504.07347*, 2025.
- [4] LMCACHE. Kv cache size calculator. https://lmcache.ai/kv_cache_calculator.html. Accessed: 2025-10-1.